

# Seguridad del sitio web

## MANUAL DE RESOLUCIÓN PASO A PASO

Unidad de Aprendizaje 4 · Proyecto **Web\_Sesion15** (ASP.NET Core Razor Pages · .NET 10 · EF Core 10) · Base de datos **VentasLeon**

Este manual reconstruye, paso a paso, la seguridad del sitio sobre el proyecto que ya traíamos de las sesiones anteriores. Al terminar, el sitio exigirá iniciar sesión, mostrará el menú según el rol del usuario, bloqueará las páginas que no le corresponden y cerrará la sesión por inactividad. Todo el código que aparece aquí está tomado tal cual del proyecto de ejemplo.

**Qué se traduce del material antiguo.** El PPT original usa Web Forms ( `aspnet_regsql.exe` , Membership/Roles de `System.Web` , `web.config` por carpeta, control Login y JScript de sesión). Nada de eso existe en ASP.NET Core. Aquí lo resolvemos con esquema propio + hash, autenticación por cookie, políticas de autorización y JavaScript de inactividad. La idea de fondo es la misma; cambia el *cómo*.

## 0 Preparar la base de datos

La seguridad se apoya en tres tablas nuevas dentro de **VentasLeon**. Como los scripts son **incrementales**, la base ya debe existir. Ejecuta los cuatro scripts de la carpeta `SQL/` del proyecto **en este orden**, una sola vez cada uno:

- 1 `VentasLeon.sql` — crea la base y las tablas de la Sesión 12.
- 2 `VentasLeon_S13.sql` — proveedores y ubigeo.
- 3 `VentasLeon_S14.sql` — productos, orden de compra y SP transaccional.
- 4 `VentasLeon_S15.sql` — roles, usuarios y la vista de seguridad (esta sesión).

Si al correr el S15 aparece «Database 'VentasLeon' does not exist» es porque falta correr primero el script de la Sesión 12. Verifica al final con una ventana nueva: `USE VentasLeon; SELECT * FROM Tb_Rol;` — deben salir **Administrador** y **Operador**.

## 1 Laboratorio 1 · Objetos de seguridad, roles y usuarios

### 1.1 · Tablas de seguridad (script `VentasLeon_S15.sql`)

En vez de las tablas `aspnet_*` que generaba el utilitario antiguo, usamos un esquema propio y simple: un rol, un usuario y la tabla puente entre ambos. La clave se guarda como **hash**, nunca en texto plano.

```

/* ----- ROL */
/* Cada rol agrupa permisos. El caso de uso maneja 2: Administrador y Operador. */
CREATE TABLE Tb_Rol (
    Cod_rol    INT          NOT NULL IDENTITY(1,1) PRIMARY KEY,
    Nom_rol    VARCHAR(30)  NOT NULL UNIQUE
);
GO

/* ----- USUARIO */
/* La clave NUNCA se guarda en texto plano: se almacena su hash (lo genera la
   aplicacion con PasswordHasher de ASP.NET Core). Est_usu: 1 activo, 0 inactivo. */
CREATE TABLE Tb_Usuario (
    Cod_usu    INT          NOT NULL IDENTITY(1,1) PRIMARY KEY,
    Login      VARCHAR(30)  NOT NULL UNIQUE,
    Clave      VARCHAR(200) NOT NULL,
    Nombre     VARCHAR(80)  NOT NULL,
    Email      VARCHAR(120) NULL,
    Est_usu    INT          NOT NULL DEFAULT 1,
    Fec_Registro DATETIME   NOT NULL DEFAULT GETDATE()
);
GO

/* ----- USUARIO-ROL */
/* Relacion muchos a muchos: un usuario puede tener uno o varios roles. */
CREATE TABLE Tb_Usuario_Rol (
    Cod_usu    INT NOT NULL REFERENCES Tb_Usuario(Cod_usu),
    Cod_rol    INT NOT NULL REFERENCES Tb_Rol(Cod_rol),
    CONSTRAINT PK_Usuario_Rol PRIMARY KEY (Cod_usu, Cod_rol)
);
GO

/* Roles base del caso de uso. Los usuarios los crea la aplicacion (hash de clave):
   al iniciar el sitio por primera vez se siembran 'admin' y 'operador' si la tabla
   de usuarios esta vacia (ver SecuritySeeder en la capa de datos). Tambien se pueden
   crear mas roles y usuarios desde la pagina Seguridad (solo Administrador). */
INSERT INTO Tb_Rol (Nom_rol) VALUES ('Administrador'), ('Operador');
GO

/* Vista de apoyo: usuario con la lista de sus roles (se usa al autenticar). */
CREATE VIEW vw_Usuario_Roles AS
    SELECT u.Cod_usu, u.Login, u.Nombre, u.Email, u.Est_usu, r.Nom_rol
    FROM   Tb_Usuario u
    JOIN   Tb_Usuario_Rol ur ON ur.Cod_usu = u.Cod_usu
    JOIN   Tb_Rol r         ON r.Cod_rol = ur.Cod_rol;
GO

```

**PROTECCIÓN**

La contraseña se define como `VARCHAR(200)` porque guardará el **hash** que produce `PasswordHasher`, no la clave real. La base nunca conoce la contraseña en claro.

## 1.2 · Entidades (carpeta Modelos)

Tres clases mapean las tablas. `Usuario` tiene una colección de roles (muchos a muchos a través de `UsuarioRol`).

```

using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;

namespace Web_Sesion15.Modelos;

// Rol de seguridad (Tb_Rol). Agrupa permisos: Administrador, Operador, etc.
[Table("Tb_Rol")]
public class Rol
{
    [Key]
    [Column("Cod_rol")]
    public int CodRol { get; set; }

    [Column("Nom_rol")]
    public string NomRol { get; set; } = "";
}

```

```

using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;

namespace Web_Sesion15.Modelos;

// Usuario del sitio (Tb_Usuario). La clave se guarda hasheada, nunca en texto plano.
[Table("Tb_Usuario")]
public class Usuario
{
    [Key]
    [Column("Cod_usu")]
    public int CodUsu { get; set; }

    [Column("Login")]
    public string Login { get; set; } = "";

    // Hash de la contraseña (lo genera PasswordHasher en la capa de datos).
    [Column("Clave")]
    public string Clave { get; set; } = "";

    [Column("Nombre")]
    public string Nombre { get; set; } = "";

    [Column("Email")]
    public string? Email { get; set; }

    [Column("Est_usu")]
    public int EstUsu { get; set; }

    [Column("Fec_Registro")]
    public DateTime FecRegistro { get; set; }

    // Roles asignados (relacion muchos a muchos a traves de Tb_Usuario_Rol).
    public List<UsuarioRol> Roles { get; set; } = new();
}

```

```
using System.ComponentModel.DataAnnotations.Schema;

namespace Web_Sesion15.Modelos;

// Tabla puente usuario-rol (Tb_Usuario_Rol). Clave compuesta (Cod_usu + Cod_rol).
[Table("Tb_Usuario_Rol")]
public class UsuarioRol
{
    [Column("Cod_usu")]
    public int CodUsu { get; set; }

    [Column("Cod_rol")]
    public int CodRol { get; set; }

    public Usuario? Usuario { get; set; }
    public Rol? Rol { get; set; }
}

```

### 1.3 · Registrar las tablas en el DbContext

Se agregan los `DbSet` y, en `OnModelCreating`, la clave compuesta y las llaves foráneas de la tabla puente (así EF Core no inventa columnas).

#### Modelos/VentasLeonContext.cs · DbSet

```
// Tablas nuevas de la Sesión 15 (seguridad del sitio: roles y usuarios)
public DbSet<Rol> Roles { get; set; }
public DbSet<Usuario> Usuarios { get; set; }
public DbSet<UsuarioRol> UsuariosRoles { get; set; }

```

#### Modelos/VentasLeonContext.cs · OnModelCreating

```
// ---- Sesión 15: seguridad (usuario - rol muchos a muchos) ----
// Clave compuesta de la tabla puente (Cod_usu + Cod_rol)
modelBuilder.Entity<UsuarioRol>()
    .HasKey(ur => new { ur.CodUsu, ur.CodRol });

modelBuilder.Entity<UsuarioRol>()
    .HasOne(ur => ur.Usuario).WithMany(u => u.Roles).HasForeignKey(ur =>
ur.CodUsu);

modelBuilder.Entity<UsuarioRol>()
    .HasOne(ur => ur.Rol).WithMany().HasForeignKey(ur => ur.CodRol);

```

### 1.4 · Gestión de seguridad en la capa de datos

Toda la lógica vive en `VentasRepositorio` (las páginas nunca tocan la BD directamente). Aquí se hashea la clave al crear el usuario y se verifica al iniciar sesión.

#### Datos/VentasRepositorio.cs · hash de la clave

```
// Instancia reutilizable para generar y verificar el hash de la clave.
private static readonly PasswordHasher<Usuario> _hasher = new();

```

```

public async Task<bool> CrearUsuarioAsync(UsuarioNuevo datos)
{
    var login = (datos.Login ?? "").Trim();
    if (login == "" || string.IsNullOrEmpty(datos.Clave))
        return false;

    if (await _ctx.Usuarios.AnyAsync(u => u.Login == login))
        return false;    // login ya usado

    var usuario = new Usuario
    {
        Login = login,
        Nombre = (datos.Nombre ?? "").Trim(),
        Email = datos.Email,
        EstUsu = 1,
        FecRegistro = DateTime.Now
    };
    // la clave se guarda como hash, nunca en texto plano
    usuario.Clave = _hasher.HashPassword(usuario, datos.Clave);
    usuario.Roles.Add(new UsuarioRol { CodRol = datos.CodRol });

    _ctx.Usuarios.Add(usuario);
    await _ctx.SaveChangesAsync();
    return true;
}

```

```

public async Task<UsuarioAutenticado?> ValidarCredencialesAsync(string login, string clave)
{
    var usuario = await _ctx.Usuarios
        .Include(u => u.Roles).ThenInclude(r => r.Rol)
        .FirstOrDefaultAsync(u => u.Login == login && u.EstUsu == 1);

    if (usuario == null) return null;

    // se compara la clave contra el hash guardado
    var resultado = _hasher.VerifyHashedPassword(usuario, usuario.Clave, clave);
    if (resultado == PasswordVerificationResult.Failed) return null;

    return new UsuarioAutenticado
    {
        Login = usuario.Login,
        Nombre = usuario.Nombre,
        Roles = usuario.Roles.Select(r => r.Rol!.NomRol).ToList()
    };
}

```

**PROTECCIÓN**

La comparación se hace con `VerifyHashedPassword` contra el hash guardado. Aunque alguien lea la tabla `Tb_Usuario`, no obtiene las contraseñas. Además solo autentica usuarios con `Est_usu = 1` (activos).

## 1.5 · Sembrar seguridad inicial

Para poder entrar la primera vez, al arrancar la app se crean los roles base y dos usuarios de ejemplo si la tabla está vacía.

```

using Web_Sesion15.Modelos;

namespace Web_Sesion15.Datos;

// Siembra seguridad inicial: si no hay usuarios, crea 'admin' y 'operador' con sus
// roles, para poder iniciar sesion la primera vez. Los roles ya vienen del script
// VentasLeon_S15.sql; si faltaran, tambien se crean aqui.
public static class SecuritySeeder
{
    public static async Task SembrarAsync(VentasRepositorio repo)
    {
        // asegura los roles base
        await repo.CrearRolAsync("Administrador");
        await repo.CrearRolAsync("Operador");

        if (await repo.ExistenUsuariosAsync()) return; // ya hay usuarios, no siembra

        int codAdmin = await repo.CodigoRolAsync("Administrador");
        int codOper = await repo.CodigoRolAsync("Operador");

        // usuario administrador (clave por defecto: Admin123)
        await repo.CrearUsuarioAsync(new UsuarioNuevo
        {
            Login = "admin",
            Clave = "Admin123",
            Nombre = "Administrador del sistema",
            Email = "admin@ventasleon.pe",
            CodRol = codAdmin
        });

        // usuario operador (clave por defecto: Oper123)
        await repo.CrearUsuarioAsync(new UsuarioNuevo
        {
            Login = "operador",
            Clave = "Oper123",
            Nombre = "Operador de ventas",
            Email = "operador@ventasleon.pe",
            CodRol = codOper
        });
    }
}

```

## 2 Laboratorio 2 · Login y reglas de acceso

Aquí está el corazón de la sesión: **dónde exactamente se protege el sitio**. En Web Forms se hacía en `web.config` de cada carpeta; en Razor Pages se centraliza en `Program.cs`.

## 2.1 · Autenticación por cookie (reemplaza a «Forms»)

### Program.cs · AddAuthentication / AddCookie

```
// Tiempo de caducidad de la sesion (en minutos), leido de appsettings.json.
int minutosSesion = builder.Configuration.GetValue<int>("Seguridad:TimeoutSesionMin");

// Autenticacion por cookie: reemplaza al viejo <authentication mode="Forms">.
builder.Services.AddAuthentication(CookieAuthenticationDefaults.AuthenticationScheme)
    .AddCookie(opciones =>
    {
        opciones.LoginPath = "/Login"; // formulario de inicio de
sesion
        opciones.LogoutPath = "/Logout";
        opciones.AccessDeniedPath = "/AccesoDenegado"; // sin permiso para el
recurso
        opciones.ExpireTimeSpan = TimeSpan.FromMinutes(minutosSesion);
        opciones.SlidingExpiration = true; // se renueva con cada
actividad
    });
```

#### PROTECCIÓN

`LoginPath` hace que cualquier intento de entrar sin sesión te mande al formulario Login; `AccessDeniedPath` es a dónde va un usuario logueado que no tiene el rol requerido. La cookie caduca según `ExpireTimeSpan`.

## 2.2 · Políticas y reglas de acceso por página/carpeta

Las políticas dicen «qué rol se necesita». Las convenciones aplican esas políticas a cada página o carpeta — es el equivalente directo de los `<allow roles>` del material antiguo.

### Program.cs · políticas por rol

```
// Politicas por rol: reemplazan a las reglas <authorization> por carpeta del web.config.
builder.Services.AddAuthorization(opciones =>
{
    opciones.AddPolicy("EsAdministrador", p => p.RequireRole("Administrador"));
    opciones.AddPolicy("AdminUOperador", p => p.RequireRole("Administrador", "Operador"));
});
```

```
builder.Services.AddRazorPages(opciones =>
{
    var c = opciones.Conventions;
    // por defecto todo el sitio exige haber iniciado sesion...
    c.AuthorizeFolder("/");
    // ...salvo estas paginas publicas
    c.AllowAnonymousToPage("/Login");
    c.AllowAnonymousToPage("/AccesoDenegado");
    c.AllowAnonymousToPage("/Error");

    // reglas por rol (como los <allow roles=...> de cada carpeta del material antiguo)
    c.AuthorizePage("/Proveedores", "EsAdministrador"); // Mantenimientos -> solo
Administrador
    c.AuthorizeFolder("/Seguridad", "EsAdministrador"); // Seguridad -> solo
Administrador
    c.AuthorizePage("/OrdenCompra", "EsAdministrador"); // Transacciones -> solo
Administrador
    c.AuthorizePage("/Reporte", "AdminUOperador"); // Consultas ->
Administrador u Operador
    c.AuthorizePage("/Consulta", "AdminUOperador");
    c.AuthorizePage("/Procedimiento", "AdminUOperador");
    c.AuthorizePage("/Ajax", "AdminUOperador");
    c.AuthorizePage("/Grafico", "AdminUOperador");
    c.AuthorizePage("/Paginacion", "AdminUOperador");
});
```

**PROTECCIÓN**

Este bloque es «en qué lado se protege». `AuthorizeFolder("/")` obliga a tener sesión en todo el sitio; las `AuthorizePage/AuthorizeFolder` restringen por rol; y `AllowAnonymousToPage` deja públicas solo Login, AccesoDenegado y Error. Aunque alguien escriba la URL a mano (p. ej. `/Seguridad/Index`) sin el rol, no entra.

## 2.3 · Activar los middlewares en el orden correcto

## Program.cs · pipeline

```
app.UseAuthentication(); // primero autentica (quien es)...
app.UseAuthorization(); // ...luego autoriza (que puede hacer)

app.MapRazorPages();
```

El orden importa: **primero** `UseAuthentication` (saber quién eres) y **luego** `UseAuthorization` (ver qué puedes hacer). Invertirlos rompe la seguridad.

## 2.4 · El formulario Login (reemplaza al control Login)

El control Login «pre programado» de Web Forms se convierte en una página normal: validamos contra la capa de datos y, si es correcto, firmamos la cookie con el nombre y un claim por cada rol.



```

public async Task<IActionResult> OnPostAsync(string? returnUrl = null)
{
    returnUrl = returnUrl;
    if (!ModelState.IsValid) return Page();

    var usuario = await _repo.ValidarCredencialesAsync(Entrada.Login, Entrada.Clave);
    if (usuario == null)
    {
        Mensaje = "Usuario o contrasena incorrectos.";
        return Page();
    }

    // se arman los datos de identidad: nombre + un claim por cada rol
    var claims = new List<Claim>
    {
        new(ClaimTypes.Name, usuario.Login),
        new("NombreCompleto", usuario.Nombre)
    };
    foreach (var rol in usuario.Roles)
        claims.Add(new Claim(ClaimTypes.Role, rol));

    var identidad = new ClaimsIdentity(claims,
    CookieAuthenticationDefaults.AuthenticationScheme);
    var propiedades = new AuthenticationProperties { IsPersistent = Entrada.Recordarme
};

    // se crea la cookie de autenticacion (equivale al FormsAuthentication de antes)
    await HttpContext.SignInAsync(
        CookieAuthenticationDefaults.AuthenticationScheme,
        new ClaimsPrincipal(identidad),
        propiedades);

    return LocalRedirect(string.IsNullOrEmpty(returnUrl) ? "/Index" : returnUrl);
}

```

**PROTECCIÓN**

Los roles del usuario se meten como `Claim(ClaimTypes.Role, ...)` dentro de la cookie. A partir de ahí, `User.IsInRole("Administrador")` y las políticas funcionan solas en todo el sitio.

## 2.5 · Menú según el rol (reemplaza al securityTrimming del SiteMap)

En el layout mostramos u ocultamos cada opción con `User.IsInRole`. Es la capa visual; la de verdad es la del paso 2.2.

```

<ul class="navbar-nav me-auto">
    <li class="nav-item"><a class="nav-link" asp-page="/Index">Inicio</a>
</li>

    @if (User.IsInRole("Administrador"))
    {
        <li class="nav-item dropdown">
            <a class="nav-link dropdown-toggle" href="#" role="button"
data-bs-toggle="dropdown">Mantenimientos</a>
            <ul class="dropdown-menu">
                <li><a class="dropdown-item" asp-page="/Proveedores">CRUD
Proveedores</a></li>
            </ul>
        </li>
    }

    @if (User.IsInRole("Administrador") || User.IsInRole("Operador"))
    {
        <li class="nav-item dropdown">
            <a class="nav-link dropdown-toggle" href="#" role="button"
data-bs-toggle="dropdown">Consultas</a>
            <ul class="dropdown-menu">
                <li><a class="dropdown-item" asp-page="/Reporte">Reporte
Excel</a></li>
                <li><a class="dropdown-item" asp-page="/Consulta">Consulta
LINQ</a></li>
                <li><a class="dropdown-item" asp-
page="/Grafico">Grafico</a></li>
                <li><a class="dropdown-item" asp-
page="/Paginacion">Paginacion</a></li>
                <li><a class="dropdown-item" asp-
page="/Procedimiento">Procedimiento</a></li>
                <li><a class="dropdown-item" asp-page="/Ajax">AJAX</a></li>
            </ul>
        </li>
    }

    @if (User.IsInRole("Administrador"))
    {
        <li class="nav-item"><a class="nav-link" asp-
page="/OrdenCompra">Transacciones</a></li>
        <li class="nav-item"><a class="nav-link" asp-
page="/Seguridad/Index">Seguridad</a></li>
    }
</ul>

<!-- Usuario actual + cerrar sesion (equivale a LoginName + LoginStatus) --
>

<span class="navbar-text vl-usuario me-3">
    Usuario: <strong>@(User.FindFirstValue("NombreCompleto") ??
User.Identity?.Name)</strong>
</span>
<a class="btn btn-sm btn-outline-light" asp-page="/Logout">Salir</a>
</div>
</div>

```

Ocultar el enlace es comodidad, no seguridad: la protección real es la del `Program.cs`. Por eso, aunque el menú no muestre «Seguridad» al operador, el intento de entrar por URL igual lo bloquea el servidor.

## 2.6 · Cerrar sesión y acceso denegado

### Pages/Logout.cshtml.cs

```
using Microsoft.AspNetCore.Authentication;
using Microsoft.AspNetCore.Authentication.Cookies;
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.RazorPages;

namespace Web_Sesion15.Pages;

// Cierra la sesion: borra la cookie de autenticacion y vuelve al Login.
public class LogoutModel : PageModel
{
    public async Task<IActionResult> OnGetAsync()
    {
        await HttpContext.SignOutAsync(CookieAuthenticationDefaults.AuthenticationScheme);
        return RedirectToPage("/Login");
    }
}
```

### Pages/AccesoDenegado.cshtml

```
@page
@{
    ViewData["Title"] = "Acceso denegado";
}

<div class="panel text-center">
    <p class="tituloForm">Acceso denegado</p>
    <p class="text-muted">
        Su usuario no tiene el rol necesario para ver esta pagina. Si cree que es un error,
        contacte al administrador del sistema.
    </p>
    <a class="btn btn-vl" asp-page="/Index">Volver al inicio</a>
</div>
```

## 3 Laboratorio 3 · Caducidad de la sesión

Reemplazamos el JScript del Masterpage por dos piezas: la cookie que caduca en el servidor y un JavaScript que avisa y cierra por inactividad en el cliente. Los tiempos se leen de `appsettings.json`.

### 3.1 · Configuración de tiempos

#### appsettings.json

```
"Seguridad": {
  "AdvertenciaSesionMin": 1,
  "TimeoutSesionMin": 2
}
```

El único requisito: `TimeoutSesionMin` mayor que `AdvertenciaSesionMin` (el aviso sale antes del cierre). El servidor ya usa este valor en `ExpireTimeSpan` (paso 2.1).

## 3.2 · Control de inactividad en el cliente

wwwroot/js/sesion-caducidad.js

```
function iniciarControlSesion(advertenciaMin, timeoutMin) {
    var msAdvertencia = advertenciaMin * 60 * 1000;
    var msTimeout = timeoutMin * 60 * 1000;
    var tAdvertencia, tCierre;

    function programar() {
        clearTimeout(tAdvertencia);
        clearTimeout(tCierre);

        // aviso: la sesion esta por expirar
        tAdvertencia = setTimeout(function () {
            var faltan = timeoutMin - advertenciaMin;
            alert("Tu sesion va a expirar en " + faltan +
                " minuto(s) por inactividad. Realiza alguna accion para mantenerla
activa.");
        }, msAdvertencia);

        // cierre: la sesion expiro, se vuelve al Login
        tCierre = setTimeout(function () {
            alert("La sesion ha expirado. Seras redireccionado a la pagina de inicio de
sesion.");
            window.location = "/Logout";
        }, msTimeout);
    }

    // cualquier actividad del usuario reinicia el conteo de inactividad
    ["mousemove", "mousedown", "keydown", "scroll", "touchstart"].forEach(function (ev) {
        document.addEventListener(ev, programar, { passive: true });
    });

    programar(); // arranca el conteo al cargar la pagina
}
```

Y en el layout se arranca con los valores de configuración:

Pages/Shared/\_Layout.cshtml · init

```
iniciarControlSesion(@Config["Seguridad:AdvertenciaSesionMin"],
@Config["Seguridad:TimeoutSesionMin"]);
```

## 4 Resumen: ¿dónde se protege cada cosa?

Mapa rápido entre el material antiguo (Web Forms) y dónde queda la misma protección en este proyecto:

| Material antiguo (Web Forms)                                   | En este proyecto (.NET 10)                                                     |
|----------------------------------------------------------------|--------------------------------------------------------------------------------|
| <code>&lt;authentication mode="Forms"&gt;</code>               | <b>Program.cs</b> — AddAuthentication + AddCookie                              |
| web.config por carpeta<br><code>&lt;allow roles=...&gt;</code> | <b>Program.cs</b> — AuthorizeFolder / AuthorizePage con políticas              |
| roleManager / Membership<br>(aspnet_regsql)                    | <b>VentasLeon_S15.sql</b> + <b>VentasRepositorio</b> (hash con PasswordHasher) |
| SiteMap + securityTrimming                                     | <b>_Layout.cshtml</b> — menú con User.IsInRole                                 |
| Control Login (pre programado)                                 | <b>Login.cshtml.cs</b> — SignInAsync con claims de rol                         |
| LoginName / LoginStatus                                        | <b>_Layout.cshtml</b> — usuario actual + botón Salir                           |
| sessionState timeout + JScript                                 | <b>appsettings.json</b> + cookie ExpireTimeSpan + <b>sesion-caducidad.js</b>   |

## 5 Probar la seguridad

Con la base lista, ejecuta el proyecto (la siembra crea los usuarios sola) y comprueba:

- 1 Entra sin sesión a cualquier URL → te redirige al **Login**.
- 2 Inicia con `admin` / `Admin123` → ves el menú completo (Mantenimientos, Consultas, Transacciones y Seguridad).
- 3 Cierra sesión e inicia con `operador` / `Oper123` → solo ves **Consultas**. Ni Mantenimientos ni Seguridad.
- 4 Como operador, escribe a mano la URL `/Seguridad/Index` → te lleva a **AccesoDenegado** (la protección del servidor, no el menú).
- 5 Deja la pantalla sin tocar: al minuto sale el aviso y al segundo minuto la sesión se cierra y vuelve al Login (valores por defecto de `appsettings.json`).

**Usuarios sembrados:** `admin` / `Admin123` (rol Administrador) y `operador` / `Oper123` (rol Operador). Desde la página **Seguridad** (solo Administrador) se pueden crear más roles y usuarios.